


KALASALINGAM
ACADEMY OF RESEARCH AND EDUCATION
(DEEMED TO BE UNIVERSITY)
Under sec. 3 of UGC Act 1956. Accredited by NAAC with "A++" Grade



# Campus Lost and Found Matcher

PROJECT ID : PRJ038

TEAM ID : T038

Project Trainer/Guide : Mr. SHIRWIN PRINCE I

Team GitHub Repository :
https://github.com/Sera0516/CampusFind_AI.git

| Student Name | Registration Number | Individual GitHub Link |
|---|---|---|
| M.Sera | 99230041099 | https://github.com/seraabi0506-pixel/Practice-code.git |
| S.Deepadharshini | 9923005245 | https://github.com/Deepa-2404/GEN-AI.git |
| S.Siva Priya | 9923005224 | https://github.com/sivapriyasri/GEN-AI.git |
| A.S.Vasima | 9923005280 | https://github.com/vasimakabeer/Practicecode.git |

## 1. Table of Contents


1. Introduction
2. Problem Statement and Objectives
3. System Architecture – RAG Pipeline
4. Implementation Details and Technology Stack
5. Results and Testing
6. Conclusion and Future Scope


## 2. Introduction

CampusFind AI is an intelligent campus lost and found platform designed to help students and campus communities report, discover, and recover lost belongings efficiently.

In traditional campus environments, lost and found information is generally shared through notice boards, social media groups, messaging applications, or informal communication. These methods make it difficult to track reported items, identify matching belongings, and connect the person who lost an item with the person who found it.

CampusFind AI addresses these challenges by providing a centralized digital platform integrated with Artificial Intelligence (AI). Users can report lost and found items, browse existing reports, manage claims, and interact with an AI-powered assistant to search for relevant items using natural language.

The platform incorporates Retrieval-Augmented Generation (RAG), semantic embeddings, vector similarity search, and Large Language Models (LLMs) to retrieve relevant campus reports and generate context-aware responses.

By combining conventional web application functionality with AI-powered information retrieval, CampusFind AI aims to make the campus lost and found process more accessible, intelligent, and efficient.

## 3. Problem Statement and Objectives

### 3.1 Problem Statement

Students frequently lose personal belongings such as laptops, mobile phones, identification cards, wallets, books, and other essential items within campus premises.

Existing lost and found management methods have several limitations:

- Lack of a centralized platform for reporting lost and found items.
- Difficulty identifying matching reports among multiple submissions.
- Dependence on manual searching and communication.
- Limited ability to understand descriptions written in different words.
- Difficulty tracking item claims and recovery status.
- Lack of intelligent assistance for searching campus reports.

A centralized and intelligent solution is therefore required to simplify the reporting, searching, matching, and recovery of lost belongings.

### 3.2 Project Objectives

The primary objectives of CampusFind AI are:

1. Develop a centralized web-based lost and found management platform.
2. Enable users to submit lost and found item reports with descriptions and relevant details.
3. Provide an organized dashboard for managing reports and claims.

4. Implement AI-powered semantic search to identify potentially matching reports.
5. Integrate a Retrieval-Augmented Generation (RAG) pipeline to retrieve relevant information from campus records.
6. Generate natural language responses using a Large Language Model.
7. Provide relevant report details alongside AI-generated responses.
8. Deploy the application on cloud infrastructure for public accessibility.

## 4. System Architecture – RAG Pipeline

### 4.1 Overview

CampusFind AI uses a Retrieval-Augmented Generation (RAG) architecture to answer user queries using information stored in the campus lost and found database.

Instead of relying entirely on the LLM's pre-trained knowledge, the system first retrieves relevant campus reports and then provides the retrieved information as context to the language model.

This approach helps generate responses grounded in actual campus reports.

### 4.2 RAG Pipeline

The RAG pipeline consists of the following stages:

**Step 1: User Query**

The user submits a natural language question through the Angular-based AI Assistant interface.

Example:

"Has anyone found a black Dell laptop near the college library?"

**Step 2: Intent Detection**

The FastAPI backend processes the query and identifies the search intent, such as:

- Lost item search
- Found item search
- General or combined search

**Step 3: Query Embedding Generation**

The query is converted into a numerical vector representation using Cloudflare Workers AI and the BGE-small-en-v1.5 embedding model.

The generated embedding contains 384 dimensions.

**Step 4: Hybrid Information Retrieval**

The system searches the PostgreSQL database using two retrieval methods:

- Semantic vector search using pgvector and cosine similarity.
- Keyword-based search using PostgreSQL text matching.

The retrieved candidates are evaluated using a hybrid scoring mechanism that considers semantic similarity, item identity, attributes, location, and text similarity.

**Step 5: Candidate Ranking and Grouping**

The retrieved reports are ranked according to their relevance. Duplicate or closely related reports are grouped while preserving their original report identifiers.

**Step 6: Context Generation**

The relevant reports are formatted and supplied as contextual information to the Gemini Large Language Model.

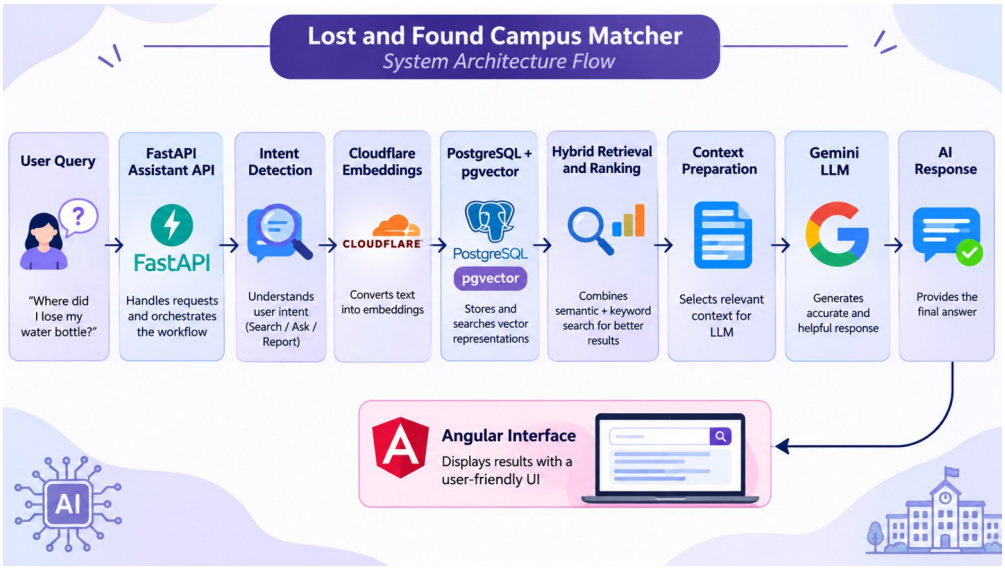
**Step 7: Response Generation**

Gemini generates a natural language response based on the retrieved campus reports.

### Step 8: Result Presentation

The generated response and relevant campus report details are returned to the Angular frontend and displayed to the user.

## 4.3 RAG Architecture Flow




## 4.4 Advantages of the RAG Approach

- Provides responses based on available campus reports.
- Combines semantic and keyword-based retrieval.
- Improves the identification of relevant items even when users use different descriptions.
- Reduces dependence on the LLM's internal knowledge for campus-specific information.
- Provides traceable report references alongside generated answers.

# 5. Implementation Details and Technology Stack

## 5.1 Frontend Development

The frontend is developed using Angular.

It provides an interactive interface through which users can access the platform's features.

Major frontend modules include:

- User dashboard.
- Lost item reporting.
- Found item reporting.
- All reports.
- My reports.
- Claims management.
- Saved items.
- Campus AI Assistant.

The AI Assistant provides a conversational interface for submitting natural language queries and viewing AI-generated responses with relevant campus reports.

### 5.2 Backend Development

The backend is implemented using Python and FastAPI.

FastAPI handles application APIs, authentication, report management, database interactions, and AI assistant requests.

The RAG implementation is organized into dedicated services for embedding generation and retrieval-augmented response generation.

Important backend components include:

- API endpoints for lost and found reports.
- Authentication and authorization using JWT.
- Database operations using SQLAlchemy.
- Embedding generation service.
- RAG retrieval and response generation service.
- AI Assistant API endpoint.

The AI Assistant exposes the endpoint:

POST /assistant/ask

It accepts a natural language question and returns the generated answer, retrieved reports, and retrieval count.

### 5.3 Database Implementation

PostgreSQL is used as the primary relational database.

The pgvector extension enables storage and similarity searching of vector embeddings.

The database supports:

- User information.
- Lost and found reports.
- Item descriptions and locations.
- Claim-related information.
- Vector embeddings for semantic retrieval.

### 5.4 Technology Stack

| Component | Technology |
|---|---|
| Frontend | Angular |
| Backend | Python, FastAPI |
| Database | PostgreSQL |
| ORM | SQLAlchemy |
| Vector Search | pgvector |
| Embedding Model | BGE-small-en-v1.5 |
| Embedding Provider | Cloudflare Workers AI |
| Language Model | Google Gemini |
| Authentication | JWT |
| Image Storage | Cloudinary |

| | |
|---|---|
| Frontend Hosting | Vercel |
| Backend Hosting | Render |
| Version Control | Git and GitHub |

### 5.5 Deployment

The application is deployed using cloud-based hosting services.

- **Frontend:** Deployed through Vercel.
- **Backend:** Deployed through Render.
- **Database:** PostgreSQL with pgvector.
- **Source Code:** Maintained in GitHub.

The frontend communicates with the deployed FastAPI backend through REST APIs.

Production environment variables are configured separately from local development settings to protect sensitive credentials and API keys.

## 6. Results and Testing

### 6.1 Implementation Results

The CampusFind AI platform has been successfully implemented and deployed.

The application provides a centralized environment for reporting and searching lost and found belongings.

The integrated RAG-based AI Assistant successfully retrieves relevant reports and generates natural language responses using the retrieved information.

### 6.2 AI Assistant Testing

A production test was performed using the following query:

**User Query:**

"Has anyone found a black Dell laptop near the college library?"

**Retrieved Result:**

- Report ID: 2
- Item Name: Dell Laptop Found
- Item Description: Black Dell laptop with silver logo.
- Location: College Library
- Report Type: Found

**Retrieval Metrics:**

| Metric | Result |
|---|---|
| Semantic Similarity | 0.8896 |
| Hybrid Relevance Score | 87.73 |
| Embedding Dimensions | 384 |
| Retrieved Reports | 1 |

The system successfully identified the matching report and generated a response through Gemini.

### 6.3 Functional Testing

| Test Case | Expected Result | Status |
|---|---|---|
| Access dashboard | Dashboard loads successfully | Passed |
| Navigate to AI Assistant | Assistant page opens | Passed |
| Submit natural language query | Query reaches backend | Passed |
| Generate query embeddings | 384-dimensional embedding generated | Passed |
| Perform semantic search | Relevant reports retrieved | Passed |
| Perform keyword search | Matching reports identified | Passed |
| Generate AI response | Gemini generates contextual | Passed |

| | | |
|---|---|---|
| | answer | |
| Display retrieved reports | Relevant report cards displayed | Passed |
| Production deployment | Application accessible through Vercel and Render | Passed |

### 6.4 Observations

During deployment testing, an unavailable Gemini model configuration initially caused response generation to fail.

The issue was resolved by updating the deployed Gemini model configuration.

Following the correction, the complete retrieval and response-generation pipeline was successfully tested in production.

The testing demonstrated that the system could retrieve a relevant report using semantic and keyword-based retrieval and present the result through the AI Assistant interface.

**Live Application:** https://campus-find-ai-phi.vercel.app

## 7. Conclusion and Future Scope

### 7.1 Conclusion

CampusFind AI successfully demonstrates the integration of Artificial Intelligence into a practical campus lost and found management system.

The platform combines Angular, FastAPI, PostgreSQL, vector search, Cloudflare Workers AI, and Google Gemini to provide an intelligent approach to reporting and recovering lost belongings.

The implementation of Retrieval-Augmented Generation enables the AI Assistant to retrieve relevant campus reports and generate contextual responses based on available information.

The application has been deployed successfully, and its core AI retrieval and response-generation workflow has been tested in the production environment.

Overall, CampusFind AI demonstrates how modern web technologies, semantic search, and Large Language Models can be combined to solve a real-world problem within an educational environment.

## 7.2 Future Scope

The platform can be further enhanced with the following features:

1. Image-Based Item Recognition

Integrate computer vision models to identify and compare uploaded images of lost and found items.

2. Multilingual AI Assistance

Support Tamil and other regional languages alongside English to improve accessibility for campus users.

3. Advanced Semantic Matching

Improve matching accuracy through more advanced embedding models, relevance feedback, and contextual ranking.

4. Real-Time Notifications

Notify users when a potentially matching lost or found report is submitted.

5. AI-Assisted Report Creation

Allow users to describe an item conversationally and automatically generate structured lost or found reports.

6. Improved Duplicate Detection

Identify duplicate reports and group similar submissions more effectively.

7. Analytics Dashboard

Provide insights into frequently lost items, common campus locations, and item recovery trends.

8. Mobile Application

Extend the platform into a dedicated mobile application for easier access and real-time campus notifications.